



Novel method for welding gantry robot scheduling at shipyards

Jongsung Lee, Byung-In Kim & Mihee Nam

To cite this article: Jongsung Lee, Byung-In Kim & Mihee Nam (2023) Novel method for welding gantry robot scheduling at shipyards, International Journal of Production Research, 61:17, 5842-5859, DOI: [10.1080/00207543.2022.2117869](https://doi.org/10.1080/00207543.2022.2117869)

To link to this article: <https://doi.org/10.1080/00207543.2022.2117869>



Published online: 06 Sep 2022.



Submit your article to this journal [↗](#)



Article views: 492



View related articles [↗](#)



View Crossmark data [↗](#)



Citing articles: 5 View citing articles [↗](#)



Novel method for welding gantry robot scheduling at shipyards

Jongsung Lee^a, Byung-In Kim^b and Mihee Nam^c

^aDepartment of Industrial and Management Engineering, Korea National University of Transportation, Chungju, Republic of Korea;

^bDepartment of Industrial and Management Engineering, Pohang University of Science and Technology (POSTECH), Pohang, Republic of Korea;

^cSamsung Heavy Industries Co., Geoje-si, Republic of Korea

ABSTRACT

Welding is the most critical operation in the shipbuilding process and has a significant influence on the production cost and quality of ships. Therefore, the welding operation must be optimised. This paper presents a real-world welding gantry robot scheduling problem at shipyards, in which three gantry robots function in parallel. Welding gantry robots cannot cross each other and should operate over a certain distance to avoid collisions. To minimise the makespan, the welding tasks given by line segments should be evenly distributed among the three gantry robots. The welding tasks assigned to each robot should be optimally sequenced to minimise the completion time, including the waiting time required to prevent collisions with neighbouring robots. In addition, long welding edges are split, and the split small length edges are assigned to the gantry robots. This paper proposes a mixed-integer linear programming model, three-stage solution approach, and variable neighbourhood search algorithm to solve this problem. Experimental tests conducted on 20 real problem instances revealed that the proposed approach can reduce the makespan by 14% on average when compared with the conventional method.

ARTICLE HISTORY

Received 12 January 2022

Accepted 17 August 2022

KEYWORDS

Welding gantry robot; scheduling; shipbuilding; heuristics; mixed-integer linear programming

1. Introduction

Welding is a major operation in the shipbuilding industry. The shipbuilding process comprises steel cutting, assembly, painting, outfitting, erection, and sea-sailing. The assembly process accounts for 48%–50% of the entire shipbuilding process and is regarded as the most critical process in shipbuilding (Jeong et al. 2020). The process is used for the fabrication of hull blocks, in which subassembly, middle assembly, and large assembly are conducted sequentially according to the assembly scale. Most ship assembly processes consist of welding numerous steel plates and stiffeners, which requires the largest amount of work in the shipbuilding process (Park, Seo, and Kang 2009). As the welding process significantly influence the production cost and quality of ships, the welding operation should be optimised.

We introduce a real-world welding gantry robot scheduling problem that occurs at shipyards, in which three gantry robots operate simultaneously. The shipyards are owned by Samsung Heavy Industries Co., which is one of the largest shipbuilders in the world. The welding tasks given by line segments should be balanced among the gantry robots, and the welding tasks assigned

to each robot should be optimally sequenced to minimise the completion time, including the waiting time required to prevent collisions with neighbouring robots. In addition, long welding edges can be split. Welding gantry robots cannot cross each other and should operate over a certain distance to avoid collisions. The makespan must be minimised.

We developed a mixed-integer linear programming model and propose a three-stage solution approach and a variable neighbourhood search (VNS) algorithm to solve the problem. Experimental results on 20 real problem instances demonstrate the effectiveness of the proposed three-stage solution approach. The proposed solution approach can reduce the makespan by 14% on average when compared with the conventional method. The average optimality gap of the solutions of the proposed algorithm compared to the lower bounds is 1.7%.

The remainder of this paper is organised as follows. Section 2 reviews the related studies. Section 3 describes the problem and the proposed mathematical model. Section 4 presents the proposed solution approach. Section 5 details the computational experiments conducted on real-world problems, followed by the conclusions in Section 6.

2. Literature review

This section reviews the related literature on the welding gantry robot scheduling problem. There are two closely related categories of research: welding sequence optimisation (WSO), and rural postman problems (RPPs).

Studies that considered WSO were mainly conducted in the automobile manufacturing or shipbuilding industries. The objectives of WSO include minimising the welding path length, deformation, and energy consumption; and determining a collision-free path if utilising multiple welding machines. Two types of welding processes, namely spot and arc welding, were considered in these studies (Wang et al. 2021).

Spot welding, in which the weld tasks are presented as a set of spots, is generally conducted in the automobile manufacturing industry. Considering that car frames are produced in production lines with multiple welding robots, such studies conducted on car manufacturing aim to minimise the makespan (cycle time) of welding robots while preventing robot collisions. Branch-and-bound and metaheuristics, including genetic algorithm (GA) and particle swarm optimisation (PSO), have been used to solve the given spot welding problems (Kovács 2016; Spensieri et al. 2016; Wang et al. 2016; Chao and Sun 2017; Xin et al. 2020).

Several studies were conducted to determine the minimum completion time of arc welding for a single robot (Kim, Norman, and Nnaji 2002; Kim, Kim, and Nnaji 2002; Kim, Kim, and Lee 2005; Dolgui and Pashkevich 2006). After modelling the given WSO as the travelling salesman problem (TSP) or the generalised travelling salesman problem (GTSP), various typical TSP and GTSP heuristics, including metaheuristics such as the GA and ant colony algorithm, were applied to generate the welding sequence. The economical sequencing of the welding operation was the only objective of these studies because a single robot was considered. Similar solution approaches such as dynamic programming, TSP heuristics and meta-heuristics were developed for the path planning of a robotic laser cutting machine (Dolgui and Pashkevich 2009; Dewil, Vansteenwegen, and Catrysse 2014; Chentsov et al. 2018).

Multiple robots were considered in some arc welding studies to determine the minimum completion time without collision. Park and Lee (2008) extended the research conducted by Kim, Kim, and Lee (2005) that is to minimise the weld completion time by considering multiple welding robots with their robot arms moving within a range from their starting points. In their study, three metaheuristics, a GA, simulated annealing (SA), and tabu search (TS), were proposed. Although the study conducted by Park and Lee (2008) is comparable to this study,

robot collisions were not considered, and a weld edge was required to be traversed exclusively by a weld robot.

The RPP is a well-known arc routing problem in which only a given subset of required arcs are traversed (Orloff 1974). The WSO can be regarded as a variant of the RPP, assuming that the weld edges correspond to the traversed arcs in the RPP (Park and Lee 2008). In particular, the WSO with multiple robots for the minimisation of the weld completion time corresponds to the min-max K-RPP (Quirion-Blais et al. 2017), in which K identical postmen cover all the required arcs with a minimum of the maximum tour length for each postman.

Several studies have been conducted on the K-RPP. Benavent et al. (2011) introduced valid inequalities that induce new facets for the min-max K-vehicles windy rural postman problem (WRPP), in which the traverse time on an arc depends on the traversal direction. Rabani, Alamdar, and Farrokhi-Asl (2016) developed a hybrid multi-objective SA for a capacitated WRPP, in which K capacitated vehicles are considered. Quirion-Blais et al. (2017) applied the min-max K-RPP to the snow ploughing problem to determine the minimum snow ploughing completion time. Fernández and Rodríguez-Pereira (2017) analysed the complexity and optimality conditions of the multi-depot RPP (MDRPP) and min-max multi-depot RPP (MM-MDRPP). In the study, mathematical models for the MDRPP and MM-MDRPP were introduced, and valid inequalities were proposed. Ahabchane, Langevin, and Trépanier (2021) addressed a routing problem based on winter road salt spreading, considering street hierarchy and demand variation, according to the weather and traffic condition. The problem was modelled as the min-max K-RPP, and an SA-based heuristic algorithm was proposed to solve the problem. Among these variants of the RPP, MM-MDRPP is the most comparable problem to our study. However, a weld line can be assigned to multiple robots, and robot collisions are explicitly considered in our study, whereas both are not considered in the MM-MDRPP. Detailed reviews of RPP and arc routing problems can be found in Corberán, Plana, and Sanchis (2015) and Corberán et al. (2021).

Among the literature we surveyed, only two studies considered multiple arc welding gantry robots in shipyards, in which the minimum makespan of the weld completion time was determined and a weld edge was assigned to multiple robots after being separated into several parts (Kang et al. 1999; Park, Seo, and Kang 2009). In the WSO of general welding robots with welding arms, two-dimensional coordinates should be considered to prevent collisions between the robots. In the WSO of gantry robots, however, only one-dimensional coordinates, which measure separation

distances between gantry robots, are considered to prevent the collisions. Kang et al. (1999) considered four arc-welding gantry robots in the subassembly process of a shipyard. In their study, a weld edge longer than a specific length was assigned to multiple robots after being separated into several parts. A heuristic algorithm was proposed that includes weld-task balancing among the welding robots, weld-task scheduling on each robot, and the determination of a collision-free schedule considering the waiting time. Whereas Kang et al. (1999) considered four gantry robots, Park, Seo, and Kang (2009) considered three gantry robots without considering their collisions. In their study, two vertical lines were plotted to divide the weld tasks into three groups such that the weld tasks among the robots were balanced. Thereafter, a GA was utilised to determine the weld sequences to minimise the makespan.

Although the fundamental manufacturing conditions of the two abovementioned studies are similar to those of the present study, we developed a mathematical model to formally describe the problem and devised a novel heuristic algorithm introducing work assignment groups (WAGs). Previous studies on welding gantry robots attempted to determine a single feasible WAG. Conversely, the proposed method generates multiple

WAGs, which is explained in Section 4. Each of them corresponds to a balanced weld task assigned to the gantry robots. As various work assignment combinations were investigated, the proposed method demonstrated a higher solution quality than those of the abovementioned studies. Furthermore, the proposed approach uses a VNS algorithm to generate diverse WAGs and improve the solution.

In addition, the proposed algorithm seeks to explicitly determine the collision free routes. Among the two studies, Kang et al. (1999) and Park, Seo, and Kang (2009), on arc welding gantry robots, only Kang et al. (1999) considered the collision of robots. Several studies on spot welding explicitly considered robot collisions (Spensieri et al. 2016; Xin et al. 2020). However, it should be noted that spot welding is fundamentally different from arc welding, that is, spot welding is a node routing problem whereas arc welding is an arc routing problem.

3. Problem description and mathematical model

This section introduces the problem in detail and presents a mathematical model for this problem.



(a) Welding factory

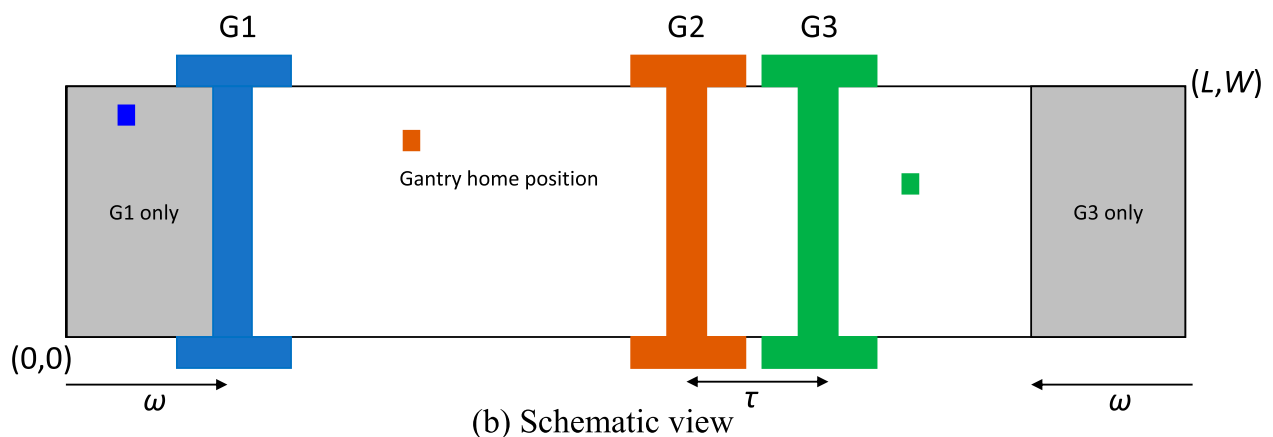


Figure 1. Welding workspace and three welding gantry robots.

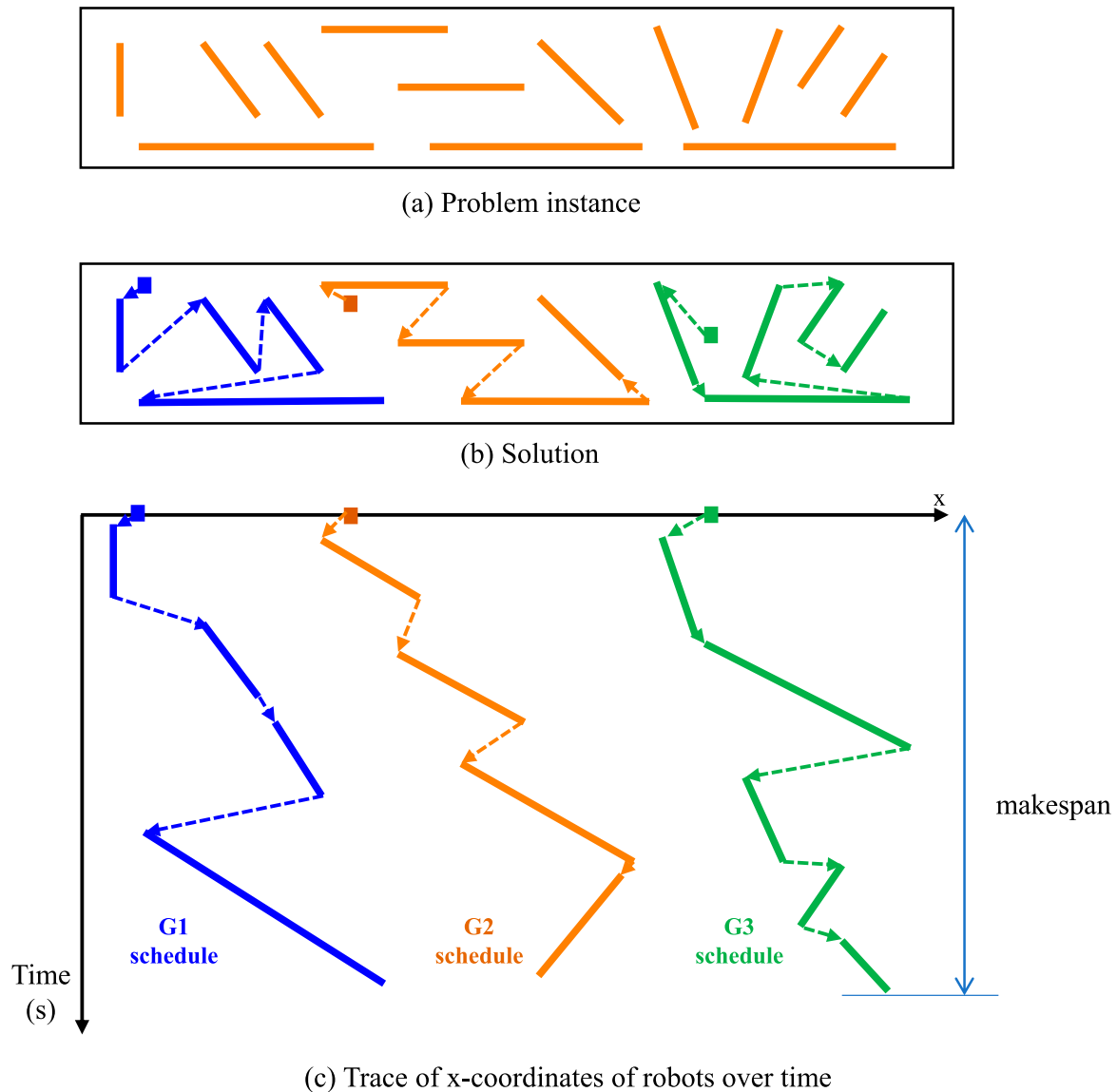


Figure 2. Sample problem instance and solution.

3.1. Problem description

The welding gantry robot scheduling problem can be expressed as follows. Figure 1(a) shows a photograph of the welding factory. There are three gantry robots (G1–G3) in a workspace with length L and width W , as shown in Figure 1(b). Welding gantry robots cannot cross paths and are required to operate over a certain distance (τ) to avoid collisions. The x-coordinates less than or equal to ω can only be accessed by G1, and the x-coordinates greater than or equal to $L - \omega$ can only be accessed by G3. The gantry robots are initiated from their home positions (small squares in Figure 1(b)) and can complete their tasks at any position.

The welding tasks are provided by line segments, as shown in Figure 2(a). A welding task can be performed in both directions, such that the start and end positions can

be selected freely. Each welding task requires a setup time (e), e.g. 30 s, before starting the task at the start position, in addition to a post-processing time (f), e.g. 20 s, after finishing the task at the end position. The actual welding time is proportional to the length of the line segment and is calculated by dividing the length by the welding speed of the robots (e.g. 10.8 mm/s). The gantry robots can move more rapidly between two positions without welding (e.g. 200 mm/s), which is referred to ‘empty travel’ in this paper.

The makespan of a given set of welding tasks is determined by the longest completion time of the robots. Thus, to minimise the makespan, the welding tasks should be assigned to the three gantry robots with a good balance, and the welding tasks assigned to each robot should be optimally sequenced to minimise the completion time. At

the same time, the waiting time required to prevent collisions with neighbourhood robots should be considered. Figure 2(b) presents an example solution for the problem instance shown in Figure 2(a). Figure 2(c) shows the trace of x-coordinates of robots over time.

3.2. Mathematical programming

To present the problem more formally, we developed a mixed integer programming model for the welding gantry robot scheduling problem. Our model is not limited to a three-robot case but can cover the problems with an arbitrary number of robots. The parameters and decision variables are as follows:

Indices and parameters

i, j = node index; node is either the welding start or end position

0 = home location of a robot

d = end location (virtual depot) of a robot

N, N^0, N^d = node set, $N + \{0\}, N + \{d\}$

r = gantry robot index, $r = 1, 2, 3, \dots, R$

t = time index (s), $t = 1, 2, 3, \dots, T$

$S = \{(i, j) | (i, j) \text{ is the required welding edge from node } i \text{ to node } j\}$

$E = \{(i, j) | (i, j) \text{ is the possible empty travel edge combination}\}$

$L_r = \{i | \text{node } i \text{ that can be assigned only to robot } r\}, r = 1, R$

p_{ij} = travel time from node i to node j (if (i, j) is given as the required weld, i.e. $(i, j) \in S$, it is the welding time. Otherwise, it is empty travel time)

d_{ij} = movement interval on x - coordinate per second (for both welding and empty travel)

τ = threshold distance between two robots

β = maximum horizontal length that a robot can travel in a second

$g(i)$ = x - coordinate of node i

e = setup time for welding

f = post processing time after welding

Decision variables

$x_{rji} = 1$, if robot r traverses node i to node j ; 0, otherwise (binary variable)

y_{ri} = arrival time of robot r at node i

$o_{rit} = |y_{ri} - t|$, difference between arrival time of robot r at node i and time t

$q_{rit} = \min(o_{rit})$, minimum value among o_{rit}

$v_{rit} = 1$, if robot r arrives at node i at time t ; 0, otherwise (binary variable)

$w_{rit} = 1$, if robot r is at node i at time t ; 0, otherwise (binary variable)

$u_{rt} = x$ - position of robot r at time t

z = makespan

The model is as follows.

$$\min z \quad (1)$$

$$\text{s.t. } \sum_r (x_{rij} + x_{rji}) = 1 \quad \forall (i, j) \in S \quad (2)$$

$$\sum_{j \in N} x_{r0j} \leq 1 \quad \forall r \quad (3)$$

$$\sum_{i \in N} x_{rid} \leq 1 \quad \forall r \quad (4)$$

$$\sum_r \sum_{j \in N^0} x_{rji} = 1 \quad \forall i \in N \quad (5)$$

$$\sum_r \sum_{j \in N^d} x_{rij} = 1 \quad \forall i \in N \quad (6)$$

$$\sum_{j \in N^0} x_{rji} = \sum_{j \in N^d} x_{rij} \quad \forall r, i \in N \quad (7)$$

$$\sum_{j \in N^0} x_{rji} = 0 \quad \forall r, i \in N \setminus L_r \quad (8)$$

$$y_{ri} + e + p_{ij} + f \leq y_{rj} + M(1 - x_{rij}) \quad \forall r, (i, j) \in S \quad (9)$$

$$y_{ri} + p_{ij} \leq y_{rj} + M(1 - x_{rij}) \quad \forall r, (i, j) \in E \quad (10)$$

$$y_{r0} = 0 \quad \forall r \quad (11)$$

$$z \geq y_{rd} \quad \forall r \quad (12)$$

$$y_{ri} \geq t - M(1 - v_{rit}) \quad \forall r, i, t \quad (13)$$

$$y_{ri} \leq t + M(1 - v_{rit}) \quad \forall r, i, t \quad (14)$$

$$\sum_t v_{rit} = \sum_{j \in N^d} x_{rij} \quad \forall r, i \quad (15)$$

$$w_{rit} \geq v_{rit} \quad \forall r, i, t \quad (16)$$

$$v_{rit} + \sum_{j \in \{j | (i, j) \in S\}} x_{rij} - w_{r,i,t+k} \leq 1 \quad \forall r, i, t, k = 0, 1, \dots, e - 1 \quad (17)$$

$$v_{rit} + x_{rij} - w_{r,j,t+e+p_{ij}+k} \leq 1 \quad \forall r, (i, j) \in S, t, k = 0, 1, \dots, f - 1 \quad (18)$$

$$\sum_i w_{rit} \leq 1 \quad \forall r, t \quad (19)$$

$$u_{rt} \geq g(i) - M(1 - w_{rit}) \quad \forall r, i, t \quad (20)$$

$$u_{rt} \leq g(i) + M(1 - w_{rit}) \quad \forall r, i, t \quad (21)$$

$$u_{r,t+e+k} \geq g(i) + kd_{ij} - M(2 - x_{rij} - v_{rit}) \quad \forall r, t, (i, j) \in S, k = 0, \dots, p_{ij} - 1 \quad (22)$$

$$u_{r,t+e+k} \leq g(i) + kd_{ij} + M(2 - x_{rij} - v_{rit}) \quad \forall r, t, (i, j) \in S, k = 0, \dots, p_{ij} - 1 \quad (23)$$

$$u_{r,t+k} \geq g(i) + kd_{ij} - M(2 - x_{rij} - v_{rit}) \quad \forall r, t, (i, j) \in E, k = 0, \dots, p_{ij} - 1 \quad (24)$$

$$u_{r,t+k} \leq g(i) + kd_{ij} + M(2 - x_{rij} - v_{rit}) \quad \forall r, t, (i, j) \in E, k = 0, \dots, p_{ij} - 1 \quad (25)$$

$$u_{r-1,t} + \tau \leq u_{rt} \quad \forall t, r = 2, 3, \dots, R \quad (26)$$

$$u_{r,t-1} - \beta \leq u_{rt} \leq u_{r,t-1} + \beta \quad \forall r, t \quad (27)$$

$$x_{rij}, v_{rit}, w_{rit} \in \{0, 1\} \quad \forall r, i, j, t \quad (28)$$

Objective function (1) minimises the makespan. Constraints (2) ensure that a welding task is assigned to a gantry robot, and one of the two directions is selected. Constraints (3) and (4) ensure that each gantry robot starts from its home position and ends at the virtual depot. Constraints (5) and (6) ensure that each node is visited. Constraints (7) represent the flow conservation condition. Constraints (8) ensure that the nodes that can be accessed by only a particular robot (the first robot, G1 or the last robot, GR) are assigned to the appropriate robot (Figure 1). Constraints (9) and (10) set the relationships between the arrival times of the two nodes of a gantry robot. Constraints (9) ensure that when (i, j) is a welding edge and a robot travels on the edge, the arrival time at node j is not less than its arrival time at node i plus the sum of the setup time, welding time between them, and post-processing time. Constraints (10) ensure that when (i, j) is an empty travel edge and a robot travels on the edge, the arrival time at node j is greater than or equal to the arrival time at node i plus the empty travel time between them. Constraints (11) initialise the arrival times at the home positions of the robots. Constraints (12) determine the makespan. These constraints are similar to those of a typical vehicle routing problem.

The remaining constraints prevent the collision of the gantry robot. Constraints (13) and (14) set the relationships between the y_{rit} and v_{rit} variables. When $v_{rit} = 1$, the arrival time of robot r at node i is t . Constraints (15) ensure that if robot r travels from node i to node j , the robot arrives at node j at one of time moments. Constraints (16) state that if a robot arrives at node i at time t , it takes the position of node i at time t . Constraints (17) ensure that when robot r arrives at node i at time t for a welding task, it remains at node i for the setup time, which is e time units. Constraints (18) ensure that when robot r arrives at node i at time t and welds through edge (i, j) , it remains at node j for the post-processing time, which is f time units, namely from $t + e + p_{ij}$ to $t + e + p_{ij} + f - 1$. Constraints (19) ensure that a robot occupies a maximum of one position of a node in a time slot. Constraints (20) and (21) set the x -position of robot r to the x -coordinate of node i when the robot occupies the position of node i at time t . Constraints (22) and (23) set the x -position of robot r along the welding edge (i, j) , and Constraints (24) and (25) set the x -position of robot r along the empty travel edge (i, j) . Constraints (26) ensure that the horizontal distance between two neighbourhood robots exceeds the threshold distance τ . Constraints (27) restrict the horizontal movement of the gantry robots within a certain range, and Constraints (28) represent binary restrictions.

3.3. Edge splitting

Several problem instances have long welding edges, as shown in Figure 3(a), and a single robot may not cover the long edges. In Figure 3(a), the shaded areas can be covered by only G1 or G3. However, as the long edge is spread over the entire workspace, it cannot be covered by a single robot. In this case, the long edges should be split and assigned to multiple robots. However, splitting a long edge is a challenging process.

In this study, we used user-specified values l_{max} and l_{min} to split the long edges. l_{max} is the threshold value by which the split-eligible edges are classified. Only the welding edges with horizontal lengths greater than l_{max} are split. l_{min} is used as the minimum length of a split sub-edge. In particular, a long welding edge is split into multiple sub-edges with a minimum horizontal length of l_{min} . Figure 3(b) presents the sub-edges split from the long edge.

However, the sub-edges must be used with care. For a normal welding edge, there is a setup time (e) and post-processing time (f). If split sub-edges are handled similar to normal welding edges, it may lead to excessive setup and post-processing times. For example, Figure 3(a) displays a long edge, which requires one setup time and one post-processing time if a robot processes the edge, although this cannot occur. In contrast, if the seven sub-edges depicted in Figure 3(b) are used, seven setup and post-processing times are counted. The correct consideration is as follows. If the first two sub-edges are assigned to G1, the subsequent three to G2, and the final two to G3, as shown in Figure 3(c), only three setup and post-processing times are counted.

The proposed solution approach presented in Section 4 considers this edge-splitting logic. This logic can be incorporated into the mathematical model presented in Section 3.2. However, the current model itself is difficult to use in real-world problem instances. Therefore, we omitted the integration of the logic.

4. Solution approach

The welding gantry robot scheduling problem contains the RPP as subproblem, which is a commonly known NP-hard problem. This paper proposes a solution approach, comprising an initial solution generation algorithm (Algorithm 1) and a variable neighbourhood search (VNS) improvement algorithm (Algorithm 2) to efficiently solve the problem.

Algorithm 1 consists of three stages: (1) the generation of WAGs, (2) generation of route sequences for each

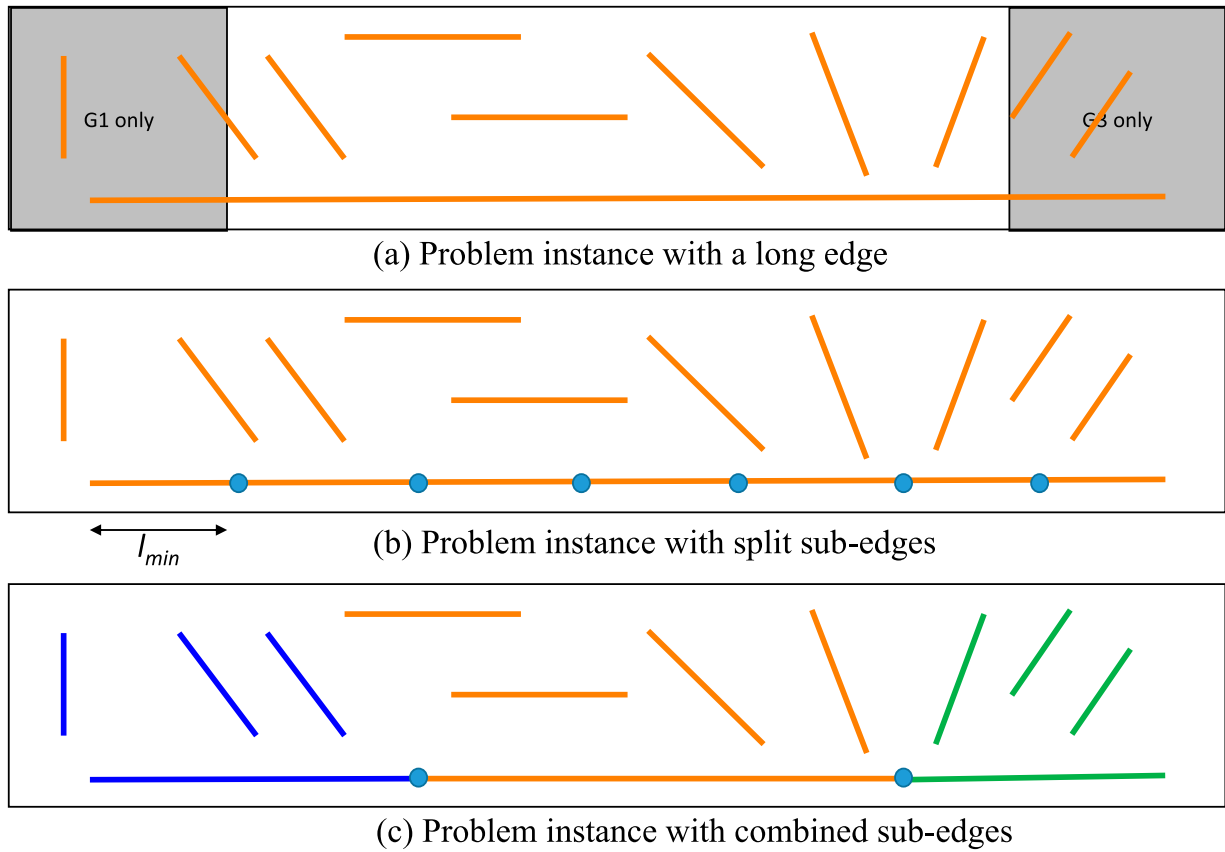


Figure 3. Problem instances considering long welding edges.

robot in each WAG, and (3) generation of a conflict-free route schedule for three routes in each WAG. In the first stage, long edges are split into small edges, and the sets of welding edges are assigned to the three robots as a WAG considering balancing. In the first stage (step 5), multiple WAGs are generated. Subsequently, the edges, which are split from the same edge and are assigned to the same robot in each WAG, are combined into an edge to prevent multiple setup and post processing times (steps 7–11). In the second stage, the assigned welding edges for each robot in each WAG are sequenced using construction and improvement algorithms (step 14). In the third stage, conflict-free route schedules are generated for all possible combinations of the generated routes in each WAG (steps 16–25), and the best route schedule with the smallest makespan is selected as the best solution (steps 20 and 21).

Algorithm 2 takes the best solution and corresponding best WAG from Algorithm 1 and improves the solution by generating neighbourhood WAGs using VNS operators. It uses the following three neighbourhood generation operators: move (step 8), swap (step 10), and large neighbourhood search (LNS) operator (step 12). The operators are randomly chosen (step 6). Then, the three stages of Algorithm 1 are processed (step 15).

Whenever the best solution is updated, the corresponding WAG becomes the best WAG, which is the base WAG from which the neighbourhood WAGs are generated. Algorithm 2 is iterated I_{max} times, which is a user-defined number, and the best solution is obtained. The detailed procedures are presented below.

4.1. Generation of WAGs

In the first stage of Algorithm 1, the welding edges with lengths larger than the user-given value l_{max} are split into small edges with a minimum length of l_{min} (step 4). For example, if $l_{max} = 5,000$ mm and $l_{min} = 1,000$ mm, an edge with a length of 6,600 mm is split into six edges, that is $\lfloor 6,600/1,000 \rfloor$, with a length of 1,100 mm. Figure 4(a) presents an example of an edge split.

Thereafter, the WAGs are generated by considering the total welding time balance among the robots as follows. First, the edges are sorted based on the leftmost x-coordinates of the edges. In the example problem of Figure 4(a), the sorted edge list is (1, 11–1, 2, 3, 11–2, 4, 5, 11–3, 11–4, 6, 11–5, 7, 8, 11–6, 9, 10). The edges that can be covered by only G1 or G3 are assigned to the appropriate gantry. In the example problem shown in Figure 4(b), the edges (1, 11–1, 2) and (11–6, 9, 10) are assigned to G1

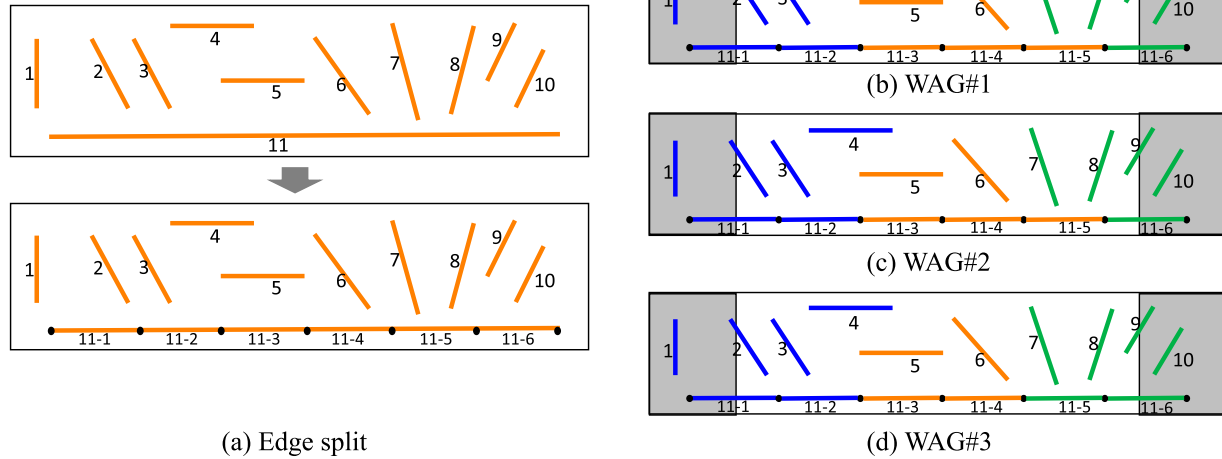


Figure 4. Generation of WAGs.

Algorithm 1. Initial solution generation algorithm

```

1: Input: welding edge set  $S = \{e | e = (i, j) \text{ is edge from node } i \text{ to node } j\}$ 
2:  $l_{max}$  = threshold horizontal length for edge split
3:  $l_{min}$  = minimum split edge length
4: Split edges whose horizontal length is larger than  $l_{max}$  considering  $l_{min}$  and generate extended edge set  $S'$ 
5: Generate work assignment groups (WAGs) by allocating edges of  $S'$  to three robots considering  $W = (w_1, w_2, w_3) | w_r = e | e \text{ is assigned to robot } r, e \in S'$  (Section 4.1)
6: check = false
7: for each WAG  $W_k$  in  $W$  do //  $W_k \rightarrow W'_k$ 
8:   for each  $w_r$  in  $W_k$  do
9:     Combine edges that split from the same edges into new sub-edges and generate  $w'_r$ 
10:   end for
11: end for
12: for each WAG  $W'_k$  in  $W'$  do
13:   for each  $w'_r$  in  $W'_k$  do
14:     Generate route sequence set  $R_r$  by construction and improvement algorithms (Section 4.2)
15:   end for
16:   for each route sequence  $r_1$  in  $R_1$  do
17:     for each route sequence  $r_2$  in  $R_2$  do
18:       for each route sequence  $r_3$  in  $R_3$  do
19:         Solution  $s = \text{Generate conflict free route schedule}$  using  $r_1, r_2, r_3$  (Section 4.3)
20:       if check = false or makespan of  $s <$  makespan of  $best\_s$  then
21:          $best\_s = s, best\_WAG = W_k, \text{check} = \text{true}$ 
22:       end if
23:     end for
24:   end for
25: end for
26: end for
27: return  $best\_s, best\_WAG$ 

```

Algorithm 2. VNS improvement algorithm

```

1: Input:  $best\_s, best\_WAG$ 
2:  $n$  = number of neighbourhoods for operators 1 and 2
3:  $p$  = percentage of solution destroy for operator 3
4:  $l_{max}$  = maximum iteration number
5: for  $m = 1$  To  $l_{max}$  do
6:   op = RandomIntNum%3 + 1 // random number from 1,2,3
7:   if op == 1 then
8:     Generate WAGs  $W$  (size =  $n$ ) by modifying  $best\_WAG$  with move operator (Section 4.4.1)
9:   else if op == 2 then
10:    Generate WAGs  $W$  (size =  $n$ ) by modifying  $best\_WAG$  with swap operator (Section 4.4.2)
11:   else if op == 3 then
12:    Generate modified WAGs  $W$  by LNS operator with  $p$  (Section 4.4.3)
13:   end if
14:   check = true
15:   steps 7–26 of Algorithm 1
16: end for
17: return  $best\_s$ 

```

times of the edge set (1, 11–1, 2, 3, 11–2, 4, 5, 11–3, 11–4, 6, 11–5, 7, 8, 11–6, 9, 10) are (300, 350, 300, 300, 350, 400, 400, 350, 350, 500, 350, 500, 500, 350, 290, 300) s, which include the setup and post-processing times, then the current welding times of G1, G2, and G3 are 950, 4,000, and 940 s, respectively.

The first or the final edge of G2 is iteratively assigned to G1 or G3 until the workload (total welding time) of G2 is the largest among the three. During the process, when the workload of G1 is smaller than that of G3, the first edge of G2 is assigned to G1. When the workload of G3 is the smallest, the final edge of G2 is assigned to G3. In the example problem, the edge sets (1, 11–1, 2, 3, 11–2), (4, 5, 11–3, 11–4, 6, 11–5), and (7, 8, 11–6, 9, 10) are assigned to G1, G2, and G3, respectively, as shown in Figure 4(b). The total welding times of G1, G2, and G3

and G3, respectively. It should be noted that the shaded areas are the exclusive areas of G1 or G3 in Figure 4(b). The remaining edges are assigned to G2. In the example case, the edges (3, 11–2, 4, 5, 11–3, 11–4, 6, 11–5, 7, 8) are assigned to G2. Thereafter, the total welding time of the edges for each group is calculated. If the welding

are 1,600, 2,350, and 1,940 s, respectively. This is the first WAG for an example problem.

The first or the final edge of G2 is iteratively assigned to G1 or G3 until the workload (total welding time) of G2 becomes the lowest among the three and generates a WAG in each iteration. In the example problem, given that the current workload of G1 is the smallest, the first edge (4) of G2 is assigned to G1. The edge sets (1, 11–1, 2, 3, 11–2, 4), (5, 11–3, 11–4, 6, 11–5), and (7, 8, 11–6, 9, 10) are assigned to G1, G2, and G3, respectively, as shown in Figure 4(c). The total welding times of G1, G2, and G3 are 2,000, 1,950, and 1,940 s, respectively. This is the second WAG for the example problem. In the following iteration, the final edge (11–5) of G2 is assigned to G3. The edge sets (1, 11–1, 2, 3, 11–2, 4), (5, 11–3, 11–4, 6), and (11–5, 7, 8, 11–6, 9, 10) are assigned to G1, G2, and G3, respectively, as shown in Figure 4(d). The total welding times of G1, G2, and G3 are 2,000, 1,600, and 2,290 s, respectively. This is the third WAG for the example problem. The iteration is then stopped because the workload of G2 is the smallest.

After generating WAGs, the split edges are combined into longer edges, as described in Section 3.3 and Figure 3. In the example cases, edges (11–1, 11–2) and (11–3, 11–4, 11–5) are combined into new edges, as shown in Figure 4(b) and (c). Edges (11–1, 11–2), (11–3, 11–4), and (11–5, 11–6) are combined into three edges, as shown in Figure 4(d), to avoid counting the setup and post-processing times more than once for the edges. Furthermore, the number of WAGs generated varies according to the problem instances.

4.2. Route sequence generation

At this stage, three route sequences are generated for the edge set assigned to a robot for each WAG (step 14 in Algorithm 1). Thus, if there are N WAGs, $3 \times 3(\text{number of robots}) \times N$ routes are generated. The route sequences are generated using two construction algorithms and two improvement algorithms.

4.2.1. Construction algorithms

The nearest addition algorithm and the farthest insertion algorithm are used to generate two routes. The route starts from the home position of the robot. Using the nearest addition algorithm, the closest edge from the home position is selected first, and the closest edge from the selected edge is thereafter selected. The procedure is conducted until all the edges are sequenced. We denote the route as $r1$.

Using the farthest insertion algorithm, the edge that has the largest x-coordinate point is routed first. Thereafter, the edge that can be inserted into the existing route

with the largest incremental distance is routed. In this procedure, each of the unrouted edges is tested to be inserted between all possible two routed edges, and its minimum incremental distance and the insertion position are obtained. Furthermore, in this procedure, the two directions of an edge are tested, and the least incremental distance, direction, and insertion position are obtained. Subsequently, the edge whose least incremental distance is the largest among all the unrouted edges is selected and inserted into the route. We denote the route as $r2$.

4.2.2. Improvement algorithms

In this study, the 2-opt and factorial edge combination algorithms are used to improve the routes generated by the construction algorithms. The two improvement algorithms are iteratively applied to each route until both algorithms cannot improve the route further.

The 2-opt algorithm attempts to replace two empty travel edges with two other empty travel edges, and accepts the replacement if it can reduce the empty travel times, as demonstrated in Figure 5(a). In this case, the directions of the welding edges and empty travel edges between the replaced edges are reversed. The algorithm attempts to replace all two possible empty travel edges and is implemented until there is no further improvement. Furthermore, the 2-opt algorithm has an option of considering the final edge. When the option is on, in addition to the regular 2-opt operations, the replacement of a single empty travel edge with the new empty travel edge that connects the starting node of the replaced edge to the end node of the final welding edge is also attempted repetitively to reduce the total routing time. Figure 5(b) presents an example of this procedure. The directions of the welding edges and empty travel edges after the replaced edge are reversed.

The proposed factorial edge combination algorithm generates all possible node sequences for a sub-route consisting of a given number of edges and selects the optimal sequence. The algorithm is implemented for all possible sub-routes until there is no further improvement. Figure 6(a) presents example sub-routes when the user-specified number of edges is 3, namely 1–2–3 and 2–3–4. For each sub-route, there are 48 ($= 3! \times 2^3$) possible node sequences. For example, for sub-route 1–2–3, there are six edge sequences (1–2–3, 1–3–2, 2–1–3, 2–3–1, 3–1–2, and 3–2–1), and each edge sequence has eight different edge start-node and end-node combinations. If the nodes of edges 1, 2, and 3 are (a, b), (c, d), and (e, f), respectively, then the possible node sequences for edge sequences 3–2–1 are e-f-c-d-a-b, e-f-c-d-b-a, e-f-d-c-a-b, e-f-d-c-b-a, f-e-c-d-a-b, f-e-c-d-b-a, f-e-d-c-a-b, and f-e-d-c-b-a. When the user-specified number of edges

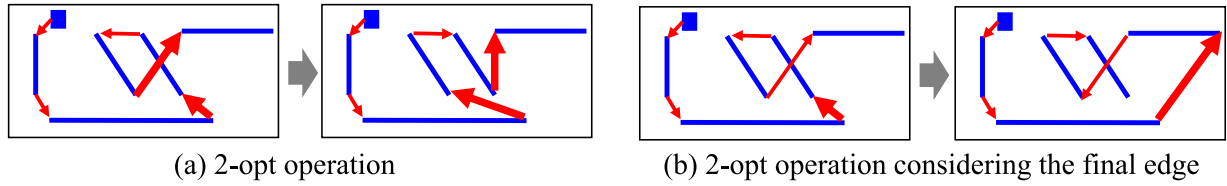


Figure 5. Two-opt algorithm.

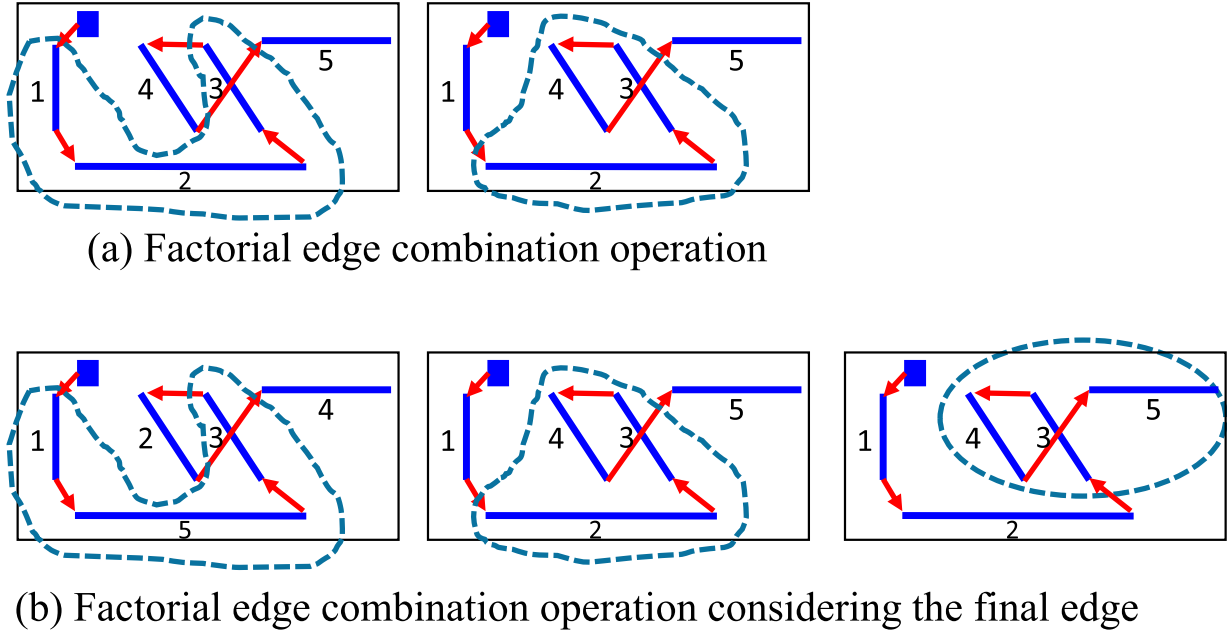


Figure 6. Factorial edge combination algorithm.

is 5, which was used for the experimental tests, there are 3,840 ($= 5! \times 2^5$) possible node sequences for each sub-route.

Similar to the 2-opt algorithm, the factorial edge combination algorithm has an option of considering the final edge. When the option is on, the final edge can be included in a sub-route, as shown in Figure 6(b). To maintain the final welding edge, the factorial edge combination algorithm without considering the final edge is used, whereas, to change all edges, the factorial edge combination algorithm with considering the final edge is used.

The route generated by the nearest addition algorithm, $r1$, is improved by the 2-opt and factorial edge combination algorithms, considering the final edge in both algorithms. We denote the improved route as i_r1 .

Meanwhile, the route generated by the farthest insertion algorithm, $r2$, is improved by the 2-opt and factorial edge combination algorithms, without considering the final edge in both algorithms. We denote the improved route as i_r2 . The route constructed by the farthest insertion algorithm, $r2$, exhibits an edge sequence trend from left to right, and the trend is maintained to increase the probability of conflict-free route schedule generation

in the following stage. However, to obtain a more efficient route sequence, the improved route i_r2 is further improved by the 2-opt and factorial edge combination algorithms, with considering the final edge. We denote the improved route as i_r3 . Thus, three route sequences, namely i_r1 , i_r2 , and i_r3 , are generated for the edge set assigned to a robot for each WAG.

4.3. Conflict-free route schedule generation

At this stage, three route sequences, each assigned to each gantry robot, are scheduled to prevent robot collisions. When conflict between two robots is imminent, which implies that the distance between two robots may be within the threshold distance τ simultaneously, one robot is required to wait until the conflict is resolved. Given that three route sequences i_r1 , i_r2 , and i_r3 are generated for the edge set assigned to a robot for each WAG, the possible number of route sequence combinations for each WAG is $3 \times 3 \times 3$. If there are N WAGs, $3 \times 3 \times 3 \times N$ route sequence combinations are considered, and the best route schedule with the minimum makespan is selected. The conflict-free route schedule generation procedure for a set of three route sequences

for a WAG is described below. It should be noted that this is one of the $3 \times 3 \times 3 \times N$ possible route sequence combinations.

Considering that $G1_r$, $G2_r$, and $G3_r$ are the target combinations of route sequences for a WAG, the schedules of $G1$, $G2$, and $G3$ are generated sequentially. Given that $G1$ is the first robot, its sequence is scheduled without considering conflict with other robots. The x-coordinate of $G1$ is traced, as shown in Figure 2(c). Thereafter, the schedule $G2_r$ for $G2$ is generated. At this stage, before $G2$ starts to move to the following point, it assesses whether the subsequent movement can be completed without conflict with $G1$. Conflict assessment can be conducted by comparing the x-positions of $G1$ and the subsequent x-positions of $G2$. If there is no conflict for the subsequent movement, $G2$ moves to the following point. Conversely, if there is conflict associated with the following movement, $G2$ waits until the conflict is resolved. The procedure is repeated until the final welding edge. Thereafter, the schedule of $G3_r$ is generated by considering the x-positions of $G2$.

If the schedules of $G1$, $G2$, and $G3$ do not have waiting times to prevent conflict, the procedure ends. However, if they have waiting times, other orders of gantry robots, namely $G3 \rightarrow G2 \rightarrow G1$ and $G2 \rightarrow G1 \rightarrow G3$, are tested, and the optimal schedule with the minimum makespan is selected for the combination.

4.4. VNS improvement algorithm

VNS is a metaheuristic algorithm proposed by Mladenović and Hansen (1997), and it has been successfully applied to various combinatorial optimisation problems (Hansen et al. 2019). VNS uses multiple neighbourhood operators to generate neighbourhood solutions. In this study, we propose a VNS with three operators to generate neighbourhood WAGs in Algorithm 2. These are move, swap, and LNS operators. LNS was developed by Shaw (1998) and successfully applied to many optimisations problems (Pisinger and Ropke 2019).

4.4.1. Move operator

Using $best_s$ and $best_WAG$, the robot, whose completion time is the longest among the three robots, is identified. Subsequently, an edge assigned to the robot is randomly selected and moved to the neighbourhood robot. If the longest completion time robot is robot1, the selected edge is moved to robot2, and a new WAG is generated. If the robot is robot2, the selected edge is moved to robot1 and robot3. In this case, two WAGs are generated. If the robot is robot3, the selected edge is moved to robot2. This procedure is iterated by n times. Thus, n WAGs or $2n$ WAGs are generated at step 8 of Algorithm 2.

4.4.2. Swap operator

Using $best_s$ and $best_WAG$, the robot, whose completion time is the longest among the three robots, is identified. Subsequently, an edge assigned to the robot is randomly selected, another edge assigned to the robot's neighbourhood is randomly selected, and the two edges are swapped. If the longest completion time robot is robot1 or robot3, its neighbourhood is robot2. If the robot is robot2, its neighbourhood is either robot1 or robot3. In this case, two WAGs are generated. This procedure is iterated n times. Thus, n WAGs or $2n$ WAGs are generated at step 10 of Algorithm 2.

4.4.3. LNS operator

LNS uses destroy and repair steps so that it keeps a partial solution and attempts to generate new different solutions. In this study, we delete all the edges from robot2 and randomly delete p percent of edges from robot1 and robot3 of $best_WAG$ and generate new WAGs using the procedure presented in Section 4.1. For example, if p is 30% and the number of edges assigned to robot1, robot2, and robot3 in $best_WAG$ are 20, 15, and 30, respectively, 6, 15, and 9 edges are deleted from robot1, robot2, and robot3. Then, the deleted 30 edges are sorted based on the leftmost x-coordinates of the edges and assigned one by one to the robots following the procedure described in Section 4.1. As the procedure described in Section 4.1 generates multiple WAGs, this LNS operator also generates multiple WAGs. The LNS operator may generate WAGs remarkably different from $best_WAG$.

5. Results

In this section, the results of the computational experiments conducted on 20 real-world problem instances are reported. The algorithms were implemented in C++. The computations were performed on a computer with an Intel® Core™ i5-6500 dual-core 3.20-GHz processor and 16 GB of random access memory (RAM). We used CPLEX version 12.9 to solve the mathematical models.

5.1. Problem instances and current practice

Twenty real-world problem instances were collected from Samsung Heavy Industries Co. Their characteristics are summarised in Table 1. The number of welding edges ranged from 27 to 89. Six instances had long welding edges that could be split. In the experiments, we used $l_{max} = 5,000$ mm and $l_{min} = 1,000$ mm. The welding speed of the gantry robots was 10.8 mm/s, and the empty travel speed was 200 mm/s. The setup time and post-processing time before and after welding were assumed to be 20 and 30 s, respectively. Given that the welding speed

Table 1. Summary of problem instances.

Problem ID	Number of welding edges	Number of splitable edges	Total welding time (s)
p1	27	2	7,471
p2	34	4	8,311
p3	43	0	8,380
p4	37	0	7,334
p5	35	0	7,158
p6	43	2	9,573
p7	59	0	11,444
p8	49	0	10,666
p9	89	0	12,134
p10	57	0	11,286
p11	57	2	11,617
p12	47	0	9,792
p13	60	0	11,661
p14	54	0	8,692
p15	59	0	12,900
p16	60	1	11,405
p17	62	0	7,396
p18	58	0	10,343
p19	55	0	10,330
p20	54	1	7,642
Average	52.0	0.6	9,776.8

was significantly lower than the empty travelling speed, the total welding time of a problem was significant to the final makespan of the problem. Table 1 presents the total welding time of each problem instance.

Table 2 presents the results of the current practice of the company. The average welding time is 9,807 s, and the average empty travel time is 452 s. Thus, the welding time dominates the completion time. The average makespan is 3,944 s.

5.2. Results of the proposed algorithm

Table 3 lists the results obtained using the proposed algorithm comprising Algorithm 1 and Algorithm 2. The average welding time was 9,815 s, average empty travel time was 259 s, and average makespan was 3,396 s. Compared with the results of the current practice, the makespan improved by 6%–29%, with an average improvement of 14%. The makespan is directly proportional to the productivity. Thus, the productivity increased by 14% by applying the proposed algorithm. The average computation time for a problem instance was 5.1 s.

Algorithm 2 requires the following three parameters: n (number of neighbourhoods for operators 1 and 2), p (percentage of solution destroy for operator 3), and I_{max} (maximum iteration number). We set $n = 10$, $p = 50\%$, and $I_{max} = 500$ for the experiments after preliminary tests with various values such as (n, p) (10, 10%), (10, 20%), (10, 30%), (10, 40%), (10, 50%), (5, 10%), (5, 20%), (5, 30%), (5, 40%), (5, 50%), and $I_{max} = 500$ and 1,000.

Figure 7 presents the gantry robot sequence for problem p7 generated by the proposed algorithm. Figure 8 presents problem p11, which has splitable welding edges. The numbers besides edges represent the order, and gantry robots are distinguished by colour.

5.3. Performance analysis of the algorithms

The mathematical model presented in Section 3.2 was applied to the problem instances. However, the instances could not be solved due to memory error. To verify the model, we generated small size problems using the problem instance p8, as shown in Table 4, namely p8_sub1, p8_sub2, and p8_sub3, which have 4, 7, and 14 edges, respectively. However, we could not obtain a solution within 3,600 s. Therefore, we increased the time unit from 1 s to 10 s by increasing the welding speed and empty travel speed by 10 times and by reducing the setup and post-processing times by 1/10. Following this modification, the number of time indices in the model decreased to 1/10, and we could solve the small size problems; the results are shown in Table 4. As shown, only the smallest problem instance could be solved optimally within 1 h.

Although the mathematical model cannot be used to obtain optimal solutions or lower bounds for real-world problem instances, its relaxed version can be used to generate lower bounds. The relaxed model consisting of objective function (1) and constraints (2)–(12) can provide valid lower bounds for the problem instances that do not have splitable edges, i.e. p3, p4, p5, p7, p8, p9, p10, p12, p13, p14, p15, p17, p18, and p19 (Table 1). The lower bound of the relaxed model for an instance is a valid lower bound for that instance. However, the lower bound of the relaxed model for an instance having splitable edges is not a valid lower bound for that instance, because the relaxed model does not split edges whereas the instance allows edge splits.

Table 5 presents the results of the relaxed model. With the computation time of 3,600 s for each instance, no instance could be solved optimally. The proposed algorithm demonstrated good performance. The gap was calculated by $(\text{makespan} - \text{lower bound})/\text{makespan}$. The average gap for Algorithm 1 and Algorithm 1 + Algorithm 2 for the instances where the lower bounds are valid was 2.9% and 1.7%, respectively. Note that the lower bounds may not be tight because they are the lower bounds for the relaxed model rather than the original model. We can conclude that the proposed algorithm can generate a near optimal solution for an instance within 6 s. Notice that the average computation time of Algorithm 1 is less than 1 s.

Furthermore, we experimented a baseline algorithm to compare the solution quality of the proposed approach.

Table 2. Results of current practice.

Problem Instance	G1				G2				G3				G1 + G2 + G3				Makespan (s)
	Welding Time(s)	Empty travel time(s)	Wait Time(s)	Completion Time(s)	Welding Time(s)	Empty travel time(s)	Wait Time(s)	Completion Time(s)	Welding Time(s)	Empty travel time(s)	Wait Time(s)	Completion Time(s)	Welding Time(s)	Empty travel time(s)	Wait Time(s)	Completion Time(s)	
p1	2,378	96	0	2,474	3,331	176	0	3,507	1,762	93	156	2,011	7,471	365	156	7,992	3,507
p2	1,823	94	0	1,917	3,291	158	0	3,449	3,622	188	0	3,810	8,736	440	0	9,176	3,810
p3	3,097	144	0	3,241	3,241	175	0	3,416	2,042	147	78	2,267	8,380	466	78	8,924	3,416
p4	2,150	96	0	2,246	2,347	118	0	2,465	2,837	101	0	2,938	7,334	315	0	7,649	2,938
p5	2,029	109	0	2,138	2,284	131	0	2,415	2,845	174	0	3,019	7,158	414	0	7,572	3,019
p6	3,386	147	0	3,533	3,117	152	0	3,269	3,258	144	108	3,510	9,761	443	108	10,312	3,533
p7	3,105	132	0	3,237	4,189	189	0	4,378	4,150	195	0	4,345	11,444	516	0	11,960	4,378
p8	3,174	182	0	3,356	3,887	197	0	4,084	3,605	162	0	3,767	10,666	541	0	11,207	4,084
p9	3,493	195	0	3,688	4,337	226	0	4,563	4,304	229	0	4,533	12,134	650	0	12,784	4,563
p10	4,207	132	0	4,339	3,570	138	0	3,708	3,509	146	0	3,655	11,286	416	0	11,702	4,339
p11	4,186	138	0	4,324	3,472	204	730	4,406	3,959	161	0	4,120	11,617	503	730	12,850	4,406
p12	3,394	159	0	3,553	2,545	112	0	2,657	3,853	161	0	4,014	9,792	432	0	10,224	4,014
p13	4,237	198	0	4,435	3,566	118	454	4,138	3,858	149	0	4,007	11,661	465	454	12,580	4,435
p14	2,916	127	0	3,043	2,829	144	243	3,216	2,947	109	0	3,056	8,692	380	243	9,315	3,216
p15	4,124	169	0	4,293	3,605	164	0	3,769	5,171	186	0	5,357	12,900	519	0	13,419	5,357
p16	3,174	147	0	3,321	3,510	189	284	3,983	4,721	156	0	4,877	11,405	492	284	12,181	4,877
p17	2,540	121	0	2,661	2,753	152	0	2,905	2,103	143	0	2,246	7,396	416	0	7,812	2,905
p18	3,562	165	0	3,727	3,361	164	434	3,959	3,420	158	0	3,578	10,343	487	434	11,264	3,959
p19	4,120	151	0	4,271	3,382	154	0	3,536	2,828	94	0	2,922	10,330	399	0	10,729	4,271
p20	3,656	204	0	3,860	2,041	92	0	2,133	1,945	91	0	2,036	7,642	387	0	8,029	3,860
Average	3,238	145	0	3,383	3,233	158	107	3,498	3,337	149	17	3,503	9,807	452	124	10,384	3,944

Table 3. Results of proposed algorithm.

Problem Instance	G1				G2				G3				G1 + G2 + G3				Make-span (s)	Make-span improvement	Computing time (s)
	Welding Time (s)	Empty travel time (s)	Wait Time (s)	Completion Time (s)	Welding Time (s)	Empty travel time (s)	Wait Time(s)	Completion Time (s)	Welding Time (s)	Empty travel time(s)	Wait Time(s)	Completion Time (s)	Welding Time (s)	Empty travel time(s)	Wait Time (s)	Completion Time(s)			
p1	2,534	39	84	2,657	2,618	71	0	2,689	2,421	50	0	2,471	7,573	160	84	7,817	2,689	23%	2.9
p2	2,879	89	55	3,023	2,967	75	0	3,042	2,819	44	0	2,863	8,665	208	55	8,928	3,042	20%	4.8
p3	2,893	61	0	2,954	2,836	87	0	2,923	2,651	101	0	2,752	8,380	249	0	8,629	2,954	14%	2.7
p4	2,477	80	0	2,557	2,444	120	0	2,564	2,413	68	0	2,481	7,334	268	0	7,602	2,564	13%	1.2
p5	2,365	82	0	2,447	2,365	79	0	2,444	2,428	53	0	2,481	7,158	214	0	7,372	2,481	18%	1.0
p6	3,226	74	0	3,300	3,258	69	0	3,327	3,190	56	0	3,246	9,674	199	0	9,873	3,327	6%	4.3
p7	3,800	107	0	3,907	3,809	97	0	3,906	3,835	86	0	3,921	11,444	290	0	11,734	3,921	10%	9.7
p8	3,563	97	0	3,660	3,498	91	0	3,589	3,605	77	0	3,682	10,666	265	0	10,931	3,682	10%	12.1
p9	4,034	126	0	4,160	4,066	111	0	4,177	4,034	121	0	4,155	12,134	358	0	12,492	4,177	8%	6.8
p10	3,773	76	0	3,849	3,738	105	0	3,843	3,775	92	0	3,867	11,286	273	0	11,559	3,867	11%	3.9
p11	3,810	64	0	3,874	3,848	89	60	3,997	3,959	78	0	4,037	11,617	231	60	11,908	4,037	8%	4.8
p12	3,280	70	0	3,350	3,233	107	14	3,354	3,279	87	0	3,366	9,792	264	14	10,070	3,366	16%	6.1
p13	3,842	88	0	3,930	3,897	104	0	4,001	3,922	78	0	4,000	11,661	270	0	11,931	4,001	10%	8.9
p14	2,916	85	0	3,001	2,906	70	0	2,976	2,870	79	0	2,949	8,692	234	0	8,926	3,001	7%	5.0
p15	4,352	93	0	4,445	4,282	105	0	4,387	4,266	91	0	4,357	12,900	289	0	13,189	4,445	17%	5.0
p16	3,749	105	0	3,854	3,798	107	0	3,905	3,858	86	0	3,944	11,405	298	0	11,703	3,944	19%	5.6
p17	2,474	89	0	2,563	2,459	84	0	2,543	2,463	102	0	2,565	7,396	275	0	7,671	2,565	12%	2.1
p18	3,488	70	0	3,558	3,435	83	0	3,518	3,420	103	0	3,523	10,343	256	0	10,599	3,558	10%	3.3
p19	3,465	77	0	3,542	3,468	79	0	3,547	3,397	88	42	3,527	10,330	244	42	10,616	3,547	17%	5.9
p20	2,656	95	0	2,751	2,621	127	0	2,748	2,566	104	0	2,670	7,843	326	0	8,169	2,751	29%	5.6
Average	3,279	83	7	3,369	3,277	93	4	3,374	3,259	82	2	3,343	9,815	259	13	10,086	3,396	14%	5.1

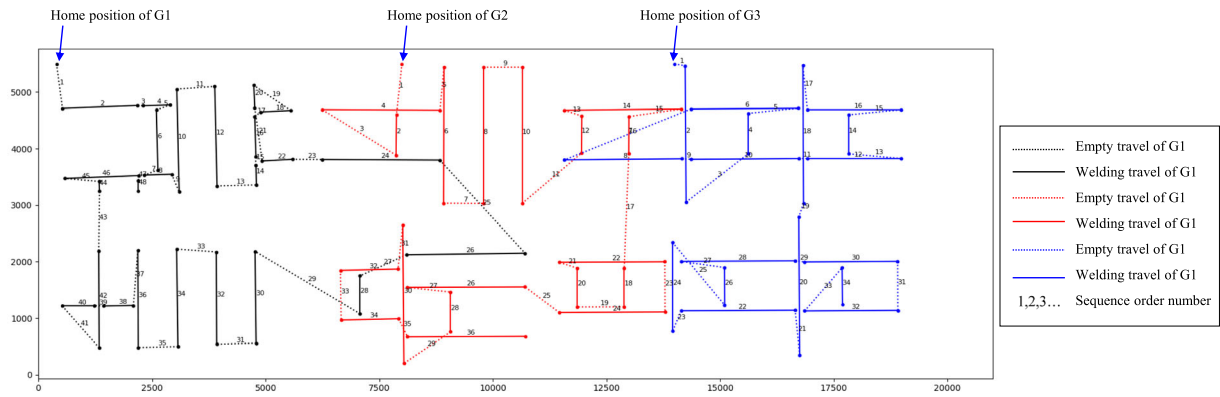


Figure 7. Gantry robot sequence generated by proposed algorithm for Problem P7.

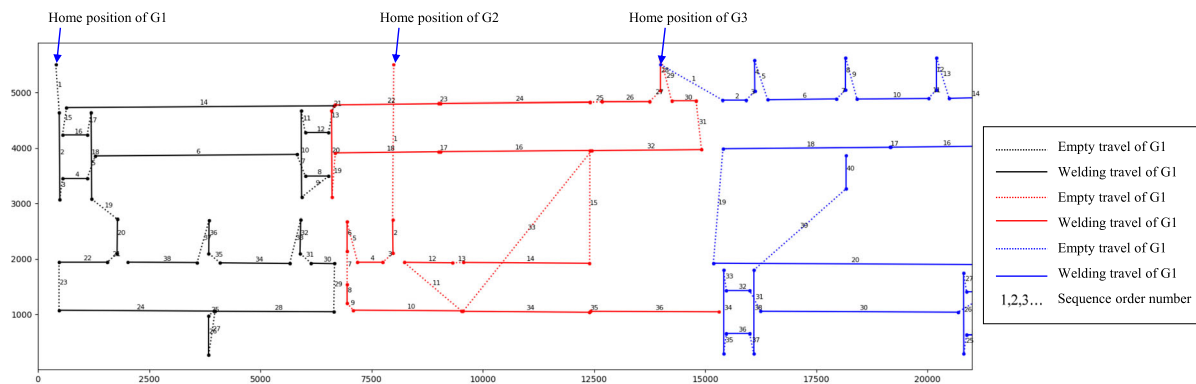


Figure 8. Gantry robot sequence generated by proposed algorithm for Problem P11.

Table 4. Results of the mathematical model with increased time unit.

Problem instance	p8_sub1	p8_sub2	p8_sub3
Number of edges	4	7	14
G1 Welding time	19	48	88
Empty travel time	8	7	11
Wait time	4	0	0
Completion time	31	55	99
G2 Welding time	29	50	78
Empty travel time	2	5	14
Wait time	0	0	7
Completion time	31	55	99
G3 Welding time	28	40	76
Empty travel time	3	4	15
Wait time	0	0	8
Completion time	31	44	99
Objective value (makespan)	31	55	99
Lower bound	31	48.7	85
Optimality gap (%)	0	11.5	14.1
Computation Time (s)	27	3,600	3,600

The baseline algorithm uses only one WAG, which is the most balanced among the generated WAGs, as well as the farthest insertion and 2-opt algorithms. In addition, a single path, $G1 \rightarrow G2 \rightarrow G3$, was employed for the conflict-free route generation procedure. The sub-procedures of the baseline algorithm are similar to those of Kang et al. (1999). Table 5 shows the performance results of the baseline algorithm. Compared with the

baseline algorithm, the proposed algorithm could find significantly better solutions (10.5% shorter makespan on average). This comparison demonstrates the effectiveness of the proposed algorithm, which uses multiple WAGs, multiple construction and improvement algorithms, numerous route combinations, multiple paths of conflict-free route generation procedures, and VNS metaheuristics.

6. Conclusions

This study addressed a real-world arc welding robot scheduling problem with three gantry robots in ship-yards. The three gantry robots work in parallel while welding a given set of weld edges located on a steel plate. A mathematical model was developed incorporating gantry robot collision avoidance constraints into the arc welding model. This study proposed a novel solution approach to minimise the maximum weld completion times of the three robots.

The proposed approach comprises an initial solution generation algorithm (Algorithm 1) and a variable neighbourhood search (VNS) improvement algorithm (Algorithm 2). Algorithm 1 has three major stages: (1) generating WAGs, (2) generating route sequences for

Table 5. Comparison results.

Problem instance	Relaxed model			Proposed algorithm				Baseline Algorithm
				Algorithm1		Algorithm1 + Algorithm2		
	Lower bound (s)	Valid LB	Upper bound (s)	Makespan (s)	Gap (%)	Makespan (s)	Gap (%)	
p1	2,720.7	N	2,764	2,771	–	2,689	–	4,602
p2	3,524.0	N	3,764	3,042	–	3,042	–	3,667
p3	2,849.8	Y	2,945	2,957	3.6%	2,954	3.5%	2,968
p4	2,480.3	Y	2,537	2,617	5.2%	2,564	3.3%	2,628
p5	2,427.7	Y	2,498	2,512	3.4%	2,481	2.1%	2,589
p6	3,492.9	N	3,556	3,399	–	3,327	–	3,492
p7	3,884.7	Y	4,020	3,962	2.0%	3,921	0.9%	4,014
p8	3,615.7	Y	3,708	3,706	2.4%	3,682	1.8%	4,693
p9	4,124.5	Y	4,354	4,240	2.7%	4,177	1.3%	4,245
p10	3,821.8	Y	3,977	3,880	1.5%	3,867	1.2%	3,904
p11	4,088.9	N	4,150	4,037	–	4,037	–	4,183
p12	3,324.8	Y	3,365	3,444	3.5%	3,366	1.2%	3,802
p13	3,947.3	Y	4,078	4,082	3.3%	4,001	1.3%	4,102
p14	2,945.4	Y	3,074	3,086	4.6%	3,001	1.9%	3,893
p15	4,370.5	Y	4,428	4,482	2.5%	4,445	1.7%	4,501
p16	3,934.6	N	4,056	3,944	–	3,944	–	4,380
p17	2,527.0	Y	2,655	2,570	1.7%	2,565	1.5%	2,590
p18	3,504.9	Y	3,639	3,633	3.5%	3,558	1.5%	3,633
p19	3,500.5	Y	3,591	3,547	1.3%	3,547	1.3%	4,622
p20	2,674.1	N	2,784	2,775	–	2,751	–	3,421
Average	3388.0		3,497.2	3,434.3	2.9%	3,396.0	1.7%	3,796.5

each robot in each WAG, and (3) generating a conflict-free route schedule for three routes in each WAG. At the first stage, long edges were split into small edges, and the sets of welding edges were assigned to the three robots as a WAG considering balancing. At the second stage, the assigned welding edges for each robot in each WAG were sequenced using two construction and two improvement algorithms. At the third stage, conflict-free route schedules were generated for all possible combinations of the generated routes in each WAG, and the best route schedule with the smallest makespan was selected as the best solution. Algorithm 2 takes the best solution and corresponding best WAG from Algorithm 1 and improves the solution by generating neighbourhood WAGs using three VNS operators.

The results of the computational tests conducted on 20 real-world problem instances revealed that the proposed algorithm significantly reduced the weld completion time by an average of 14% when compared with the results of the conventional method used by Samsung Heavy Industries Co. The average computation time is 5.1 s on a desktop computer. The proposed algorithm can obtain a near optimal solution, with an optimality gap of 1.7% on average.

The main contributions of this study are as follows: (i) We developed a mathematical programming model for the gantry robot scheduling problem at shipyards. (ii) We developed a three-stage initial solution generation approach comprising generation of WAGs, route sequencing for each robot in each WAG, and generation of a conflict-free route schedule. (iii) We developed

a VNS improvement algorithm with three neighbourhood generation operators. (iv) The generation of multiple WAGs is a new concept. (v) Customised heuristic algorithms were developed considering the nature of the problem. (vi) The proposed approach was implemented as a software system and pilot tested at Samsung Heavy Industries Co., South Korea.

There are several opportunities to improve or extend this research. First, although we developed a mathematical model, it requires a long computation time. An efficient exact approach can be pursued. Second, many routes and their combinations were tested to determine an appropriate conflict-free schedule in the proposed approach. However, WAG generation and route scheduling were separated. If an efficient algorithm that simultaneously considers both can be developed, an improved solution can be obtained. Third, the proposed algorithm may divide a long weld edge into an excessive number of pieces. A novel method for the division of a long weld edge into the exact number of gantry robots from the start of the algorithm is required to reduce the search space of route sequencing for each robot.

Data availability statement

The data that support the findings of this study are available from the corresponding author, B.-I. Kim, upon reasonable request.

Disclosure statement

No potential conflict of interest was reported by the author(s).

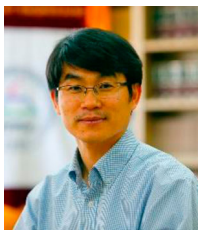
Funding

This work was supported by National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) [grant numbers 2022R1A2C2006665 and 2022R1A4A1019071].

Notes on contributors



Jongsung Lee is an Assistant Professor in the Department of Industrial and Management Engineering at Korea National University of Transportation (KNUT), Korea. He received his B.S. and Ph.D. (Integrated Master & Ph.D.) degrees in Industrial and Management Engineering from Pohang University of Science and Technology (POSTECH), Korea, in 2005 and 2015, respectively. He was a senior researcher and engineer in the Division of Device Solutions (Semiconductor) at Samsung Electronics, Korea. He is a member of KIIE and KORMS. His research interest includes bin packing problems, vehicle routing problems, and OHT (Overhead Hoist transport) routings.



Byung-In Kim is a Professor and the Head of the Department of Industrial and Management Engineering at POSTECH, Korea. He received his B.S. and M.S. degrees in Industrial Engineering from POSTECH, Korea, and Ph.D. degree in Decision Sciences and Engineering Systems from Rensselaer Polytechnic Institute, Troy, New York, in 1991, 1994, and 2002, respectively. He was an Assistant Professor of Industrial and Systems Engineering at the University of Memphis and the Director of Research and Development at Institute of Information Technology, Inc., The Woodlands, Texas. His research interests include vehicle routing problems, industrial optimization problems, traffic signal optimization, and logistics. He received the 2004 Franz Edelman Finalist Award from INFORMS, 2013 Hyun-Woo Management Science Award from the Korean Operations Research and Management Science Society, and 2018 POSCO Smart Innovation Award from POSCO. He is an associate member of the National Academy of Engineering of Korea, fellow of Asia Pacific Industrial Engineering and Management Society, and vice president of the Korean Institute of Industrial Engineers. He has more than 65 papers published in international journals such as International Journal of Production Research, European Journal of Operational Research, and IIE Transactions.



Mihee Nam is a senior engineer at Samsung Heavy Industries, Korea. She received her B.S. degree in Mechanical Design Engineering from Chungnam National University, Korea, in 2001 and M.S. degree in Mechanical Engineering from Yonsei University, Korea, in 2003. She received her Ph.D. in the Robotics Related Interdisciplinary Course from Pusan National University, Korea, in 2017. She received the Engineer of Korea Award from the Ministry of Science and ICT, Korea, in 2018. Her research interests include robotic control and automatic system control.

References

- Ahabchane, Chahid, André Langevin, and Martin Trépanier. 2021. "Robust Optimization for the Hierarchical Mixed Capacitated General Routing Problem Applied to Winter Road Maintenance." *Computers and Industrial Engineering* 158 (April). doi:10.1016/j.cie.2021.107396.
- Benavent, Enrique, Angel Corberán, Isaac Plana, and José M. Sanchis. 2011. "New Facets and an Enhanced Branch-and-Cut for the Min-Max K-Vehicles Windy Rural Postman Problem." *Networks* 58 (4): 255–272. doi:10.1002/net.20469.
- Chao, Yongsheng, and Wenlei Sun. 2017. "Motion Planning and Simulation of Multiple Welding Robots Based on Genetic Algorithm." *Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)* 10464 LNAI: 193–202. doi:10.1007/978-3-319-65298-6_18.
- Chentsov, Alexander G., Pavel A. Chentsov, Alexander A. Petunin, and Alexander N. Seseikin. 2018. "Model of Megalopolises in the Tool Path Optimisation for CNC Plate Cutting Machines." *International Journal of Production Research* 56 (14): 4819–4830. doi:10.1080/00207543.2017.1421784.
- Corberán, Ángel, Richard Eglese, Geir Hasle, Isaac Plana, and José María Sanchis. 2021. "Arc Routing Problems: A Review of the Past, Present, and Future." *Networks* 77 (1): 88–115. doi:10.1002/net.21965.
- Corberán, Ángel, Isaac Plana, and José María Sanchis. 2015. "Chapter 6: The Rural Postman Problem on Directed, Mixed, and Windy Graphs." In *Arc Routing*, 101–127. Philadelphia, PA: Society for Industrial and Applied Mathematics. doi:10.1137/1.9781611973679.ch6.
- Dewil, Reginald, Pieter Vansteenwegen, and Dirk Cattrysse. 2014. "Construction Heuristics for Generating Tool Paths for Laser Cutters." *International Journal of Production Research* 52 (20): 5965–5984. doi:10.1080/00207543.2014.895064.
- Dolgui, Alexandre, and Anatol Pashkevich. 2006. "Cluster-Level Operations Planning for the out-of-Position Robotic Arc-Welding." *International Journal of Production Research* 44 (4): 675–702. doi:10.1080/00207540500219460.
- Dolgui, Alexandre, and Anatol Pashkevich. 2009. "Manipulator Motion Planning for High-Speed Robotic Laser Cutting." *International Journal of Production Research* 47 (20): 5691–5715. doi:10.1080/00207540802070967.
- Fernández, Elena, and Jessica Rodríguez-Pereira. 2017. "Multi-Depot Rural Postman Problems." *Top* 25 (2): 340–372. doi:10.1007/s11750-016-0434-z.
- Hansen, Pierre, Nenad Mladenović, Jack Brimberg, and José A. Moreno Pérez. 2019. "Variable Neighborhood Search." In *Handbook of Metaheuristics*, 57–97. Cham: Springer. doi:10.1007/978-3-319-91086-4_3.
- Jeong, Dongsu, Dohyun Kim, Taihun Choi, and Yoonho Seo. 2020. "A Process-Based Modeling Method for Describing Production Processes of Ship Block Assembly Planning." *Processes* 8 (7). doi:10.3390/pr8070880.
- Kang, Jangha, Kyungchul Park, Sungsoo Park, and Kisang Do. 1999. "Scheduling of Welding Robots for Shipyard Sub-Assembly Process." *Journal of the Korean Institute of Industrial Engineers* 25 (1): 21–34.
- Kim, H. J., Y. D. Kim, and D. H. Lee. 2005. "Scheduling for an Arc-Welding Robot Considering Heat-Caused Distortion."

- Journal of the Operational Research Society* 56 (1): 39–50. doi:10.1057/palgrave.jors.2601803.
- Kim, Kyoung Yun, Dong Won Kim, and Bart O. Nnaji. 2002. “Robot Arc Welding Task Sequencing Using Genetic Algorithms.” *IIE Transactions (Institute of Industrial Engineers)* 34 (10): 865–880. doi:10.1080/07408170208928918.
- Kim, Kyoung Yun, B. A. Norman, and B. O. Nnaji. 2002. “Heuristics for Single-Pass Welding Task Sequencing.” *International Journal of Production Research* 40 (12): 2769–2788. doi:10.1080/00207540210139051.
- Kovács, András. 2016. “Integrated Task Sequencing and Path Planning for Robotic Remote Laser Welding.” *International Journal of Production Research* 54 (4): 1210–1224. doi:10.1080/00207543.2015.1057626.
- Mladenović, N., and P. Hansen. 1997. “Variable Neighborhood Search.” *Computers & Operations Research* 24 (11): 1097–1100. doi:10.1016/S0305-0548(97)00031-2.
- Orloff, C. S. 1974. “A Fundamental Problem in Vehicle Routing.” *Networks* 4 (1): 35–64. doi:10.1002/net.3230040105.
- Park, S. H., and D. H. Lee. 2008. “Sequencing Algorithms for Multiple Arc-Welding Robots Considering Thermal Distortion.” *International Journal of Production Research* 46 (17): 4751–4767. doi:10.1080/00207540600898056.
- Park, Ju-yong, Jeong-jin Seo, and Hyun-jin Kang. 2009. “Optimization of Robot Welding Process of Subassembly Using Genetic Algorithm in the Shipbuilding.” *Journal of the Korean Welding and Joining Society* 27 (2): 57–62. doi:10.5781/KWJS.2009.27.2.057.
- Pisinger, David, and Stefan Ropke. 2019. “Large Neighborhood Search.” In *Handbook of Metaheuristics*, 99–127. Boston, MA: Springer. doi:10.1007/978-3-319-91086-4_4.
- Quirion-Blais, Olivier, André Langevin, Fabien Lehuédé, Olivier Péton, and Martin Trépanier. 2017. “Solving the Large-Scale Min-Max K-Rural Postman Problem for Snow Plowing.” *Networks* 70 (3): 195–215. doi:10.1002/net.21759.
- Rabbani, Masoud, Safoura Famil Alamdar, and Hamed Farrokhi-Asl. 2016. “Capacitated Windy Rural Postman Problem with Several Vehicles: A Hybrid Multi-Objective Simulated Annealing Algorithm.” *International Journal of Supply and Operations Management* 2 (4): 1003–1020.
- Shaw, Paul. 1998. “Using Constraint Programming and Local Search Methods to Solve Vehicle Routing Problems.” In *International Conference on Principles and Practice of Constraint Programming. Lecture Notes in Computer Science*, 417–431. Springer Berlin Heidelberg. doi:10.1007/3-540-49481-2_30.
- Spensieri, Domenico, Johan S. Carlson, Fredrik Ekstedt, and Robert Bohlin. 2016. “An Iterative Approach for Collision Free Routing and Scheduling in Multirobot Stations.” *IEEE Transactions on Automation Science and Engineering* 13 (2): 950–962. doi:10.1109/TASE.2015.2432746.
- Wang, Xuewu, Yingpan Shi, Dongyan Ding, and Xingsheng Gu. 2016. “Double Global Optimum Genetic Algorithm-Particle Swarm Optimization-Based Welding Robot Path Planning.” *Engineering Optimization* 48 (2): 299–316. doi:10.1080/0305215X.2015.1005084.
- Wang, Xuewu, Xin Zhou, Zelong Xia, and Xingsheng Gu. 2021. “A Survey of Welding Robot Intelligent Path Optimization.” *Journal of Manufacturing Processes* 63 (2019): 14–23. doi:10.1016/j.jmapro.2020.04.085.
- Xin, Jianbin, Chuang Meng, Frederik Schulte, Jinzhu Peng, Yanhong Liu, and Rudy R. Negenborn. 2020. “A Time-Space Network Model for Collision-Free Routing of Planar Motions in a Multirobot Station.” *IEEE Transactions on Industrial Informatics* 16 (10): 6413–6422. doi:10.1109/TII.2020.2968099.